DIGITAL PROJECT REPORT

Intelligent Multilingual AGV for Grid Navigation and Parking via OCR

Group Members: Ali Guliyev, Veronika Rybak, Ruslan Tsibirov, Denis Hoti
Date of Submission: 08.07.2025
Supervisor: Prof. Dr. Pirmin Fontaine
Course Title: Digital Project in SCM, Logistics and Operation Research

Table of Contents

1. Introduction **3**
1.1 Background and Motivation 3
1.2 Objective of the Project 3
1.3 Relevance to Logistics 4
2. Technical Overview **4**
2.1 Physical System Description 4
2.2 Components Used 4
2.3 Assembly Process and Challenges 5
3. Software Design & Algorithms 6
3.1 Programming Environment 6
3.2 RoboPro Logic: Machine States and Command Flow 6
3.2.1 Line Following State 6
3.2.2 Command Handling State 8
3.3 Parking via Python Backend and OCR 9
3.4. Obstacle Avoidance Logic 10
3.5. Voice Commands and GPT Integration 11
3.6. Architecture Diagram 12
4. Integration and Testing 12
4.1. Testing Setup and Methodology 12
4.2. Key Challenges 12
4.2.1. Synchronization vs. Asynchronous Execution 12
4.2.2. Reliable Detection of the Letter "P" Using OCR 13
5. Conclusion **14**
5.1. Summary of Achievements 14
5.2. Limitations 14
5.3. Suggestions for Future Work 15
5.4. Cons and Pros 15
5.5. Team Reflections and Feedback 15
6. Appendices **16**

1. Introduction

1.1 Background and Motivation

The progression of Industry 4.0 has shifted the focus of manufacturing and logistics from manual work to smart and automated systems. One of the most important technologies in this shift is the Autonomous Guided Vehicle (AGV). AGVs are used in many tasks such as moving items between different parts of a warehouse, helping with inventory management, and even supporting basic warehouse monitoring. A well-known example is Amazon Robotics, which uses different types of mobile robots like Hercules, Pegasus, and Xanthus. These robots move shelves with products, transport packages across large fulfillment centers, and automatically avoid obstacles. They work as part of a smart logistics system that helps Amazon speed up deliveries, reduce mistakes, and improve safety in their warehouses. This shows how AGVs can play a key role in modern logistics and real-life industrial environments.

Our motivation to pursue this topic lies in its complexity and opportunity for innovation. Compared to fixed production-line automation, AGVs allow greater creative and algorithmic freedom. This flexibility enables integration with modern AI tools, the use of sensor-based dynamic control, and multimodal input interpretation—features that are at the frontier of smart logistics. Thus, AGVs represent a fertile ground for both academic exploration and industrial applicability.

1.2 Objective of the Project

The central goal of our project is the development of a smart, self-navigating AGV that can interact with users through voice commands in multiple languages, autonomously follow a color-coded grid, and identify and move toward a designated parking location using OCR (Optical Character Recognition) techniques. *OCR is a method used to extract text from images—in our case, it helps the robot detect the letter "P" on signs to find parking spots.* The AGV's behavior is driven by sensor input, real-time server communication, and algorithmic control.

We also sought to simulate an industrial use case where AGVs are deployed in warehouses or production facilities, automating both navigation and task execution in a semi-autonomous fashion. Beyond these traditional environments, AGVs are now commonly used in **city logistics**, especially for **last-mile delivery** in urban areas and campuses.

For instance, **Starship Technologies** has deployed over **2,000 sidewalk delivery robots** across more than 100 cities, completing more than **8 million autonomous deliveries** by early 2025. Their robots reliably transport food, groceries, and small parcels, navigate around pedestrians, and cross streets—often with no human onboard. Meanwhile, **Kiwibot** operates in multiple U.S. university campuses, offering "corner-to-corner" autonomous delivery services and advancing toward fully unsupervised navigation through complex environments.

These real-world applications highlight how AGVs are extending beyond warehouse floors into public urban spaces, demonstrating their versatility in smart logistics systems.

1.3 Relevance to Logistics

The relevance of AGVs in logistics is well-documented, especially in industries such as automotive manufacturing, e-commerce fulfillment, and pharmaceuticals. In such contexts, AGVs are used to autonomously transport materials across facilities, manage warehouse shelving systems, and improve delivery cycles. Our AGV prototype, though simplified, captures core functionalities such as grid navigation, dynamic task assignment, and responsive behavior to voice and visual stimuli.

For example, in logistics hubs like Amazon's fulfillment centers, AGVs are used to carry bins of products to human or robotic pickers. Similarly, in automotive plants, AGVs transport parts between assembly zones, reducing reliance on manual forklifts. Beyond warehouse environments, AGVs are also playing a role in **urban logistics**—companies such as **Starship Technologies** and **Kiwibot** deploy small autonomous delivery robots on sidewalks to handle last-mile deliveries in cities and campuses. These use cases highlight how AGVs contribute across different logistics layers, from large-scale industrial settings to customer-facing operations.

Additionally, the modularity of our AGV system makes it adaptable to various logistics needs—be it for inventory movement, parcel sorting, or zone-specific task execution—highlighting the practical potential of this educational prototype.

2. Technical Overview

2.1 Physical System Description

The core of our system is based on the Fischer Technik TXT 4.0 Controller platform, which offers both mechanical flexibility and programmable interfaces for educational robotics. The physical AGV includes several key components that enable it to perceive its surroundings, receive instructions, and move autonomously:

- **Front-mounted camera** for capturing images used in parking detection via OCR.
- **Color sensor** that detects surface markings to guide the AGV along a predefined path.
- **Distance sensor** used to detect and avoid obstacles or walls.
- **Infrared track sensor** used to identify black line on the white grid.

These components work in tandem to interpret both visual and auditory inputs and convert them into navigational decisions. The body of the AGV was designed to accommodate stable movement and sensor alignment, ensuring accuracy and reproducibility of behavior.

2.2 Components Used

All structural and electronic parts were provided within the Fischer Technik Robotics kit, specifically designed for laboratory and educational settings. This included:

- Modular chassis parts for flexible vehicle design
- Motor actuators compatible with TXT controller ports
- Color and ultrasonic sensors pre-configured for Fischertechnik environment
- Mounts and brackets to attach the camera and sensors in fixed positions

Thanks to the detailed hardware documentation provided by Fischertechnik, assembly was relatively straightforward. However, sensor placement still required iterative calibration to achieve optimal performance.

Picture 1. Assembled AGV.

2.3 Assembly Process and Challenges

The physical assembly of the AGV was guided by the manuals provided in the Fischer Technik TXT 4.0 set. While this offered a reliable reference for constructing the chassis and sensor layout, several practical issues arose during implementation.

First, although no custom camera mount was required, care had to be taken to position the camera in a way that ensured stable orientation and an unobstructed field of view. Sensor symmetry and alignment were particularly important for consistent navigation behavior, especially for the line-following and obstacle detection routines.

One of the more technical hardware challenges involved the power connection: the wire connection to the battery was initially unstable and prone to intermittent contact. We resolved this by enlarging the wire nose attachment to ensure a firmer and more consistent physical connection.

Additionally, although the color sensor did not require explicit calibration, ensuring its placement and orientation aligned correctly with the track was critical for reliable performance. The grid used for navigation was provided as part of the Fischer Technik logistics pack and required no additional construction.

Overall, while the kit simplified many aspects of the build, attention to detail and iterative testing were still essential to achieve a robust and reproducible setup.

3. Software Design & Algorithms

3.1 Programming Environment

The AGV was built using a hybrid architecture combining low-level motor and sensor control in **RoboPro Coding** and high-level behavior logic implemented in **Python**. RoboPro was responsible for real-time control loops, line following, and sensor event reactions on the Fischer Technik TXT 4.0 Controller. Python modules were executed on a local device that handles voice command parsing, multilingual GPT-assisted interpretation, OCR-based parking detection, obstacle navigation logic, and the most important returning correct command or set of commands to the TXT 4.0 to take the respective action.

Communication between RoboPro and the Python backend was realized using the **MQTT protocol**, leveraging a public broker (`broker.hivemq.com`) for all message exchanges.

Note: As `broker.hivemq.com` is a public service, the demand and the availability changes so the user might need to use other popular services to execute **MQTT** commands.. One of the popular ones is `test.mosquitto.org`.

3.2 RoboPro Logic: Machine States and Command Flow

At startup, the AGV sets its default state to “Following Line” and initializes both its motors and sensor modules. The main control loop in RoboPro continuously checks the current state of the vehicle.

3.2.1 Line Following State

During the **Line-Following** state, the AGV continuously reads inputs from the infrared track sensors. If one sensor detects it has deviated from the line, the corresponding motor slows down to re-align the vehicle — as shown in the RoboPro logic in **Figure 1**.

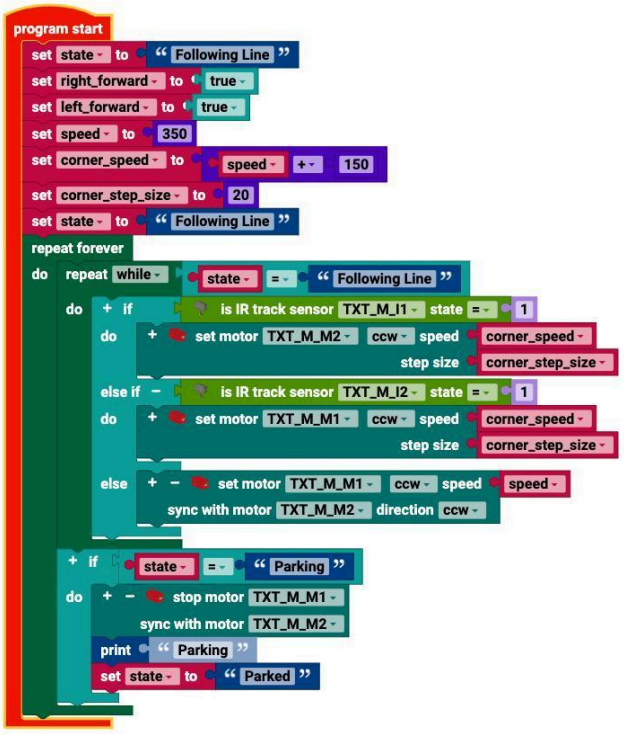

Figure 1. RoboPro visual program for "Line Following" and transition to "Parking" state.

To help clarify this process, **Figure 2** presents the same logic in simplified pseudocode:

```
Start program
Set initial speed and direction settings
Set state to "Following Line"

Repeat forever:
    If state is "Following Line":
        If left sensor sees the line (value = 1):
            Slightly slow down the right motor to turn left
        Else if right sensor sees the line (value = 1):
            Slightly slow down the left motor to turn right
        Else:
            Move both motors forward with default speed

    If state is "Parking":
        Stop both motors
        Print "Parking"
        Change state to "Parked"
```

Figure 2. Simplified pseudocode of the AGV behavior during the Line-Following and Parking states.

3.2.2 Command Handling State

When a valid command—such as “go forward,” “stop,” “turn around,” or “park”—is received from the server, RoboPro changes the AGV’s operational **state** to execute the intended action. Simple navigation commands are directly mapped in RoboPro blocks, while the “park” command triggers a specialized procedure.

Figure 3 illustrates the RoboPro framework handling state-dependent commands:

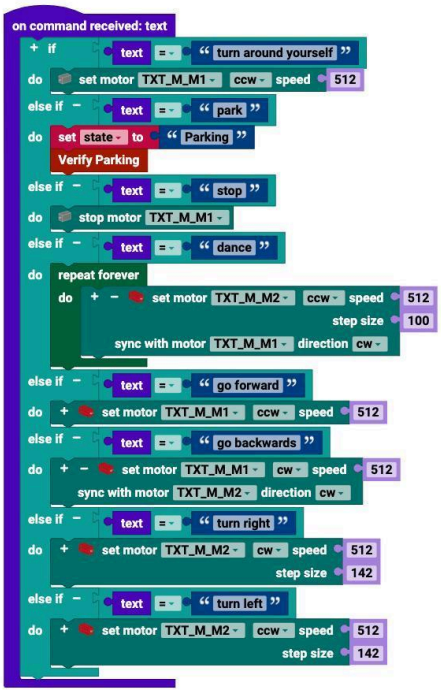

Figure 3. ***RoboPro visualization of the “on command received” logic block****, showing how different text commands map to motor actions and state transitions (including the “Park” trigger)*

Below is simplified pseudocode representing this behavior:

```
Listen for command text
If text == "turn around yourself":
    rotate both motors in opposite directions (speed=512)
Else if text == "park":
    state = "Parking"
    call VerifyParking()
Else if text == "stop":
    stop motors immediately
Else if text == "dance":
    perform continuous dance sequence (motor patterns)
Else if text == "go forward":
    move forward (speed=512)
Else if text == "go backwards":
    move backward (speed=512)
```

```
Else if text == "turn right":
    turn right (speed=512, step size=142)
Else if text == "turn left":
    turn left (speed=512, step size=142)
```

Figure 4. Simplified pseudocode of the RoboPro command-handling logic for voice commands.

By using state variables and an efficient event loop, our system allows **immediate response to new commands**—such as stopping ongoing actions and switching to the parking workflow—without waiting for current motions to complete.

3.3 Parking via Python Backend and OCR

The `Verify Parking` procedure establishes a TCP–MQTT connection to the server. **MQTT (Message Queuing Telemetry Transport)** is a lightweight messaging protocol that runs over TCP and is widely used in IoT systems for reliable communication between devices. In our setup, the AGV captures a series of images using the TXT 4.0's USB camera and sends them (base64-encoded) over MQTT to the topic `"txt4/image"`.

The backend listens to this topic and performs parking detection using OCR. It analyzes the received images to determine whether a parking spot marked with the letter **"P"** is visible. If so, it calculates the appropriate motion required to align the vehicle with the parking marker.

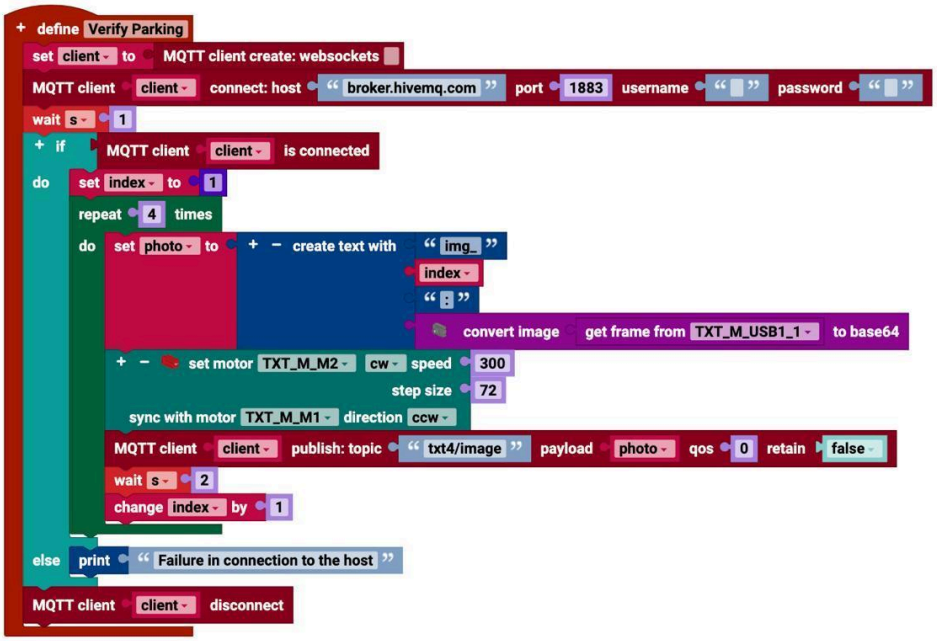

Figure 5. RoboPro "Verify Parking" procedure illustrating MQTT image capture and publishing logic.

Below is a simplified pseudocode explaining the backend's algorithm for determining the parking direction:

```
Algorithm: Determine Parking Direction
Input: Images from front camera
Output: Motor command (direction, speed, step)

1. For each image in image folder:
    a. Preprocess image (resize, grayscale, threshold)
    b. Detect contours and extract text via OCR
    c. If letter 'P' found and is largest so far → mark as best match

2. If a best match is found:
    a. If centered → move forward gently
    b. If off to the side → rotate to align
    c. Else → move forward quickly
3. Publish the motor command over MQTT
```

Figure 6. Simplified pseudocode of the OCR-driven parking decision algorithm.

Once the best parking image is selected, a motor command is returned to the AGV via the MQTT topic `"txt4/action"` in JSON format. The vehicle then interprets the JSON and executes the appropriate parking maneuver.

3.4. Obstacle Avoidance Logic

Obstacle handling is implemented primarily in RoboPro to bypass simple obstacles, but for more advanced decisions the AGV calls a backend Python script named `obstacle_handlign.py`. Upon detecting an obstacle, the vehicle captures an image of its environment and sends it to the backend via MQTT. **MQTT**, as previously mentioned, enables reliable communication between devices using a publish–subscribe model.

Based on the object's position and size in the image, the backend evaluates whether the vehicle should move forward, turn left, or turn right to avoid the obstacle. This decision is then published as a **motor command sequence** on the `"txt4/action"` topic, which the AGV listens to and executes in real time. To ensure precision and safety, the robot uses a built-in acknowledgment mechanism to confirm the completion of each command before proceeding.

```
Algorithm: Decide Obstacle Bypass Direction
Input: Image from camera
Output: Direction (left / right / forward)

1. Convert image to grayscale
2. Apply Gaussian blur and threshold
3. Find largest contour
4. Compute bounding box center
5. If object is far or small → return 'forward'
```

```
6. If object is left of center → return 'right'
7. Else → return 'left'
```

Figure 7. Pseudocode describing the decision-making algorithm for obstacle bypass logic.

3.5. Voice Commands and GPT Integration

One of the most innovative aspects of this project is the integration of **natural language commands** in multiple languages. The AGV listens via the local device's microphone, transcribes speech using the `speech_recognition` library, and uses a **fine-tuned GPT-4.1 nano model** to understand the intent and extract structured commands. This approach enables greater flexibility in user interaction and significantly reduces API costs (fallback to GPT-4o is available if needed).

The `speech_to_command.py` script is responsible for:

- Detecting and identifying the language of the command.
- Using OpenAI or Coqui TTS to respond to the user in the same language.
- Parsing the GPT response and extracting **JSON commands** (e.g., direction, speed, step size), which are then published via MQTT to the `"txt4/action"` topic.

The AGV only proceeds to execute a plan after the user **explicitly confirms** it by saying a trigger word like "Execute."

```
Algorithm: Voice Command Interpreter
Input: User speech
Output: JSON action commands

1. Record and transcribe user speech
2. Detect language; switch if needed
3. Send text prompt to GPT model
4. Parse GPT reply to extract JSON command(s)
5. Ask user to confirm ("execute")
6. If confirmed → publish each command via MQTT
```

Figure 8. Pseudocode describing the logic of voice command interpretation and GPT-based translation to motor instructions.

This layered approach enhances the AGV's usability by enabling human–machine interaction in natural speech, supporting multilingual input, and decoupling speech logic from motor control through clean command packaging.

3.6. Architecture Diagram

The full control loop can be summarized as:

1. **Start** → AGV initializes hardware and enters default line-following mode.
2. **Voice input** is received → interpreted → mapped to high-level intent.
3. **Command dispatched** to RoboPro or Python backend.
4. **RoboPro acts** on internal commands; **Python handles** OCR or logic and replies via MQTT.
5. AGV **moves, parks, or reroutes** based on result.

4. Integration and Testing

4.1. Testing Setup and Methodology

We conducted multi-phase testing to evaluate the AGV's performance across all major subsystems:

- **Phase 1:** Basic path-following on the provided Fischer Technik grid using the infrared sensors
- **Phase 2:** Voice command reception and response using the **Fischer Technik "Voice Control" app**, connected to our Python-GPT backend for interpretation
- **Phase 3:** OCR-based parking detection triggered by voice command
- **Phase 4:** Obstacle detection using visual input and autonomous bypass logic
- **Phase 5:** Sequence testing — issuing commands rapidly in succession or under environmental variation

Each test was repeated under controlled and altered conditions to assess robustness, responsiveness, and timing behavior.

4.2. Key Challenges

4.2.1. Synchronization vs. Asynchronous Execution

A major challenge in the integration phase was handling the **synchronization of commands**. Initially, commands like `"Go forward"` or `"Turn"` were executed using synchronous logic, meaning the robot would finish the full movement before checking for any new instructions. This made the system **unresponsive to follow-up or urgent commands** like `"Stop"` or `"Park"` during ongoing actions.

This issue was most noticeable in cases like:

- The user saying `"Park"` while the robot was still executing a `"Go forward"` routine. The robot would continue to move instead of starting the parking sequence immediately.

- Emergency-like situations where `"Stop"` was issued but the AGV didn't respond until its previous steps completed.

Solution: State-Based Command Switching

To address this, we implemented a **state-driven architecture in RoboPro**, using a global variable (`state`) evaluated in a high-frequency loop (`repeat forever`). This allowed us to:

- **Continuously monitor the current state** and switch behavior accordingly.
- Ensure that **any state change triggered by a voice command (e.g., `"Parking"`) is acted upon immediately**, overriding whatever task was running before.

This structure gave us an asynchronous feel while preserving the safety and structure of a synchronous control system — a vital improvement for time-sensitive interactions in real-world logistics applications.

Performance Evaluation

The integrated system showed reliable performance in real-time control and interpretation tasks:

- Voice commands, handled via the "Voice Control" app + GPT backend, were recognized and interpreted with reasonable speed.
- Parking detection worked consistently when the "P" sign was clearly visible.
- State transitions were smooth, and interruptions like `"Stop"` effectively halted previous actions without delay.

Metric	Value
Voice Command Recognition Delay	~ 2 seconds (average)
OCR Processing Time (4 Images)	~ 3.5 seconds (total)

4.2.2. Reliable Detection of the Letter "P" Using OCR

Another central challenge emerged in the parking logic — specifically, the accurate recognition of the letter **"P"** from camera images to identify designated parking zones.

Initially, we experimented with several OCR libraries, including standard options such as **Tesseract** and lightweight alternatives. However, these tools frequently failed to deliver consistent performance in our setup. Common issues included:

- **Misidentification of the letter "P"**, especially when fonts varied or lighting conditions caused distortion
- **False positives** due to noise, contour misclassification, or image blur during movement
- **Thresholding problems**, where minor changes in contrast or angle led to wildly different OCR outputs

These inconsistencies made it difficult to rely on OCR output for precise parking decisions — a critical part of our system's autonomous behavior.

To overcome this, we invested time in:

- **Selecting an OCR engine optimized for real-world camera input** — ultimately settling on **PaddleOCR**, which supported angle classification and showed superior performance under variable conditions
- **Testing under different lighting setups and camera distances**
- **Tuning internal thresholds** like bounding box size, contrast preprocessing, and match confidence to reduce noise and improve consistency

This iterative fine-tuning significantly improved our ability to detect the letter "P" reliably across all four captured frames during a parking command. While the process was time-intensive, it proved essential to ensuring our AGV could make **accurate and autonomous decisions** based on its environmental perception — a fundamental capability in any logistics application.

5. Conclusion

5.1. Summary of Achievements

Through this project, we successfully designed and implemented an autonomous guided vehicle (AGV) that simulates real-world logistics behavior using multimodal interaction — voice, vision, and sensor control. The AGV can respond to multilingual commands, detect environmental cues such as obstacles or parking signs, and make intelligent decisions in real time by integrating both on-device RoboPro logic and cloud-based Python services.

Among our key achievements:

- Seamless communication between voice input, GPT-based intent recognition, and movement execution
- Reliable obstacle bypass and parking logic based on visual perception
- A robust hybrid software architecture using MQTT, state-based control, and modular task execution

What distinguishes our AGV is its **responsiveness and adaptability**, achieved by addressing deep integration issues such as synchronous execution limitations and real-world OCR variability. Our iterative engineering process allowed us to deliver a prototype that closely reflects the automation needs of future logistics environments.

5.2. Limitations

Despite its strong functional scope, the system is still constrained by:

- **Hardware limitations**, such as low-resolution imaging and basic sensor calibration
- **No built-in parallel processing**, which makes real-time voice interaction depend on external processing speed

- **Incomplete quantitative testing** for long-term reliability, path error margins, or power efficiency

However, these challenges closely reflect what real industrial systems encounter during early-stage development.

5.3. Suggestions for Future Work

To expand the capabilities of this system, we propose:

- **Local voice and OCR processing** using a Raspberry Pi or Jetson Nano for reduced latency
- **Reinforcement learning approaches** to help the AGV learn optimal navigation and parking behavior over time
- **Grid mapping and localization**, enabling autonomous rerouting and zone recognition without user commands

---

5.4. Cons and Pros

We need to mention that as always there is another side of the coin (some drawbacks) of optimization problems such as **identification of letter 'P' to park** and as well **GPT-4.1 nano model fine-tuning**.

Identification of letter 'P': While we have tried to change the threshold value to identify 'P' correctly even if it is not fully visible, in some occasions it results with extracting different letters such as 'S' and 'R' from different images even when there is no explicit letter but some lines and figures. For now it is not an issue for our logic and can be handled in more advanced devices with higher level hardware.

GPT-4.1 nano model fine-tuning: Firstly we have started to implement the logic and create the correct pipeline with GPT-4o model, however, it was expensive in the long run so we have decided to implement cheaper but fine-tuned **GPT-4.1 nano** model. Generally it can be said that trained lower models can beat more powerful but untrained models in the specific task, this is the reason why we have ended up in **GPT-4.1 nano**.

5.5. Team Reflections and Feedback

We collectively believe that the complexity and interdisciplinary nature of this project offered valuable learning in embedded systems, machine learning, and automation. The integration of voice AI, visual logic, and mechanical control demonstrated the richness of intelligent logistics systems — and highlighted how much value thoughtful software design can add to classical robotics.

6. Appendices

- **Detailed architecture diagram**